



Namma-Santhe Ledger

Technical Documentation Report

Android • Kotlin

Jetpack Compose

MVVM + Clean Architecture

Offline-First

Firebase

Gemini OCR

QR Verification

Prepared for Academic & Project Evaluation — May 2026

Table of Contents

1. System Overview	7. App Navigation Flow
2. System Architecture	8. OCR Pipeline
3. UML Class Diagram	9. QR Confirmation System
4. Use Case Diagram	10. Security Architecture
5. Sequence Diagram	11. ER Diagram
6. Data Flow Diagram	12. Project Structure
	13. Technology Stack

1. System Overview

Problem Statement

Rural vendors in Indian markets maintain handwritten ledgers for credit tracking, leading to lost records, calculation errors, and no backups.

Solution

An **offline-first Android app** that digitizes credit tracking with automatic balance calculation, QR confirmation, OCR scanning, cloud backup, and PDF export.

Key Features

Feature	Description	Technology
Quick Entry	Numeric keypad for <5 sec entry	Jetpack Compose
Customer Mgmt	Photo capture, search, profiles	CameraX, Coil
Credit Tracking	Auto-calculated balances	Room DB
QR Confirmation	Tamper-proof verification	ZXing, HMAC-SHA256
OCR Bill Scan	Handwritten/printed bill extraction	Gemini ML + Tesseract
Cloud Sync	Firebase Auth + Firestore	Firebase
PDF Export	Shareable ledger reports	iText7
Offline Mode	Full functionality offline	Room + SQLite

2. System Architecture

4-layer **MVVM + Clean Architecture** with dependency flow top-to-bottom.

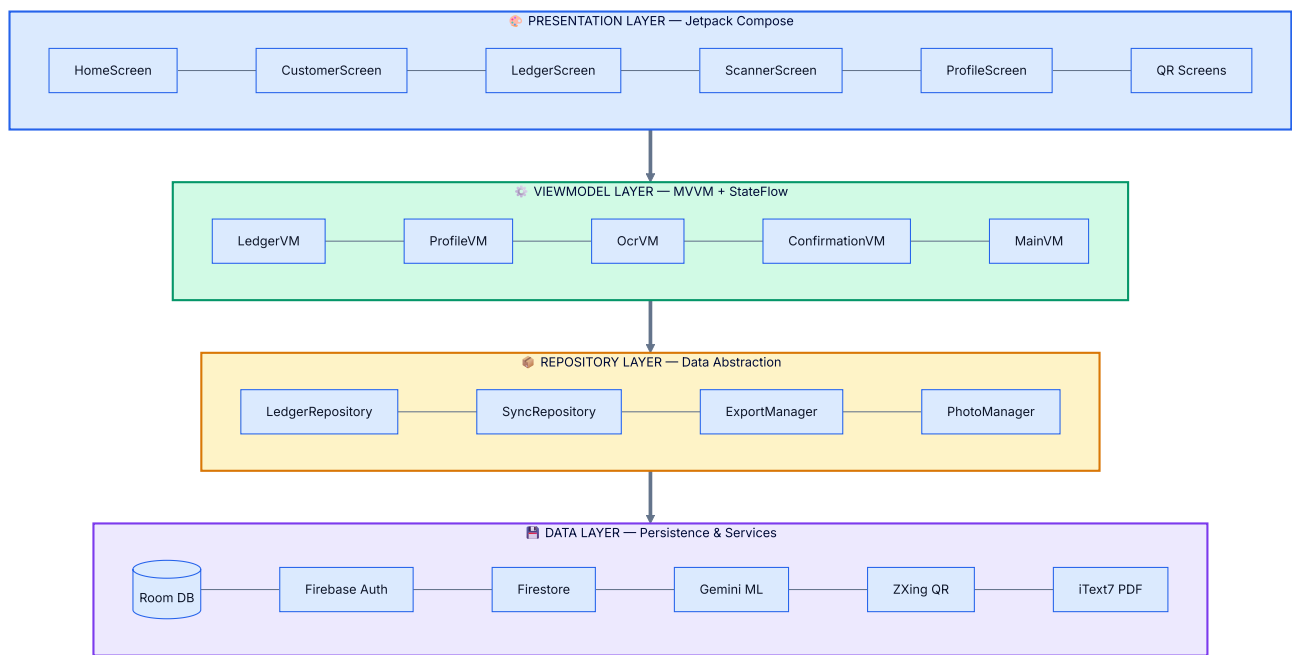
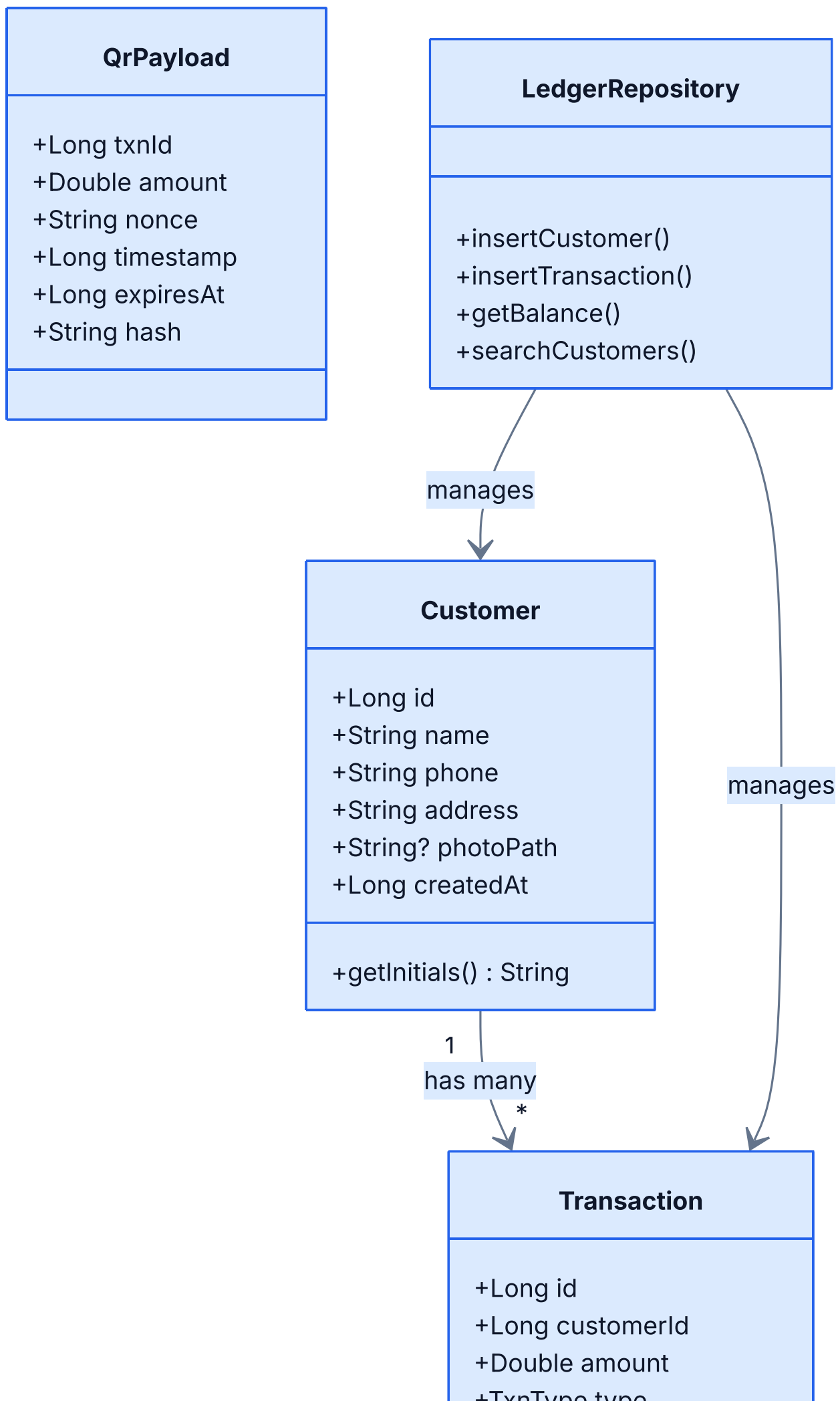


Figure 2.1 — Four-layer system architecture with dependency flow

3. UML Class Diagram



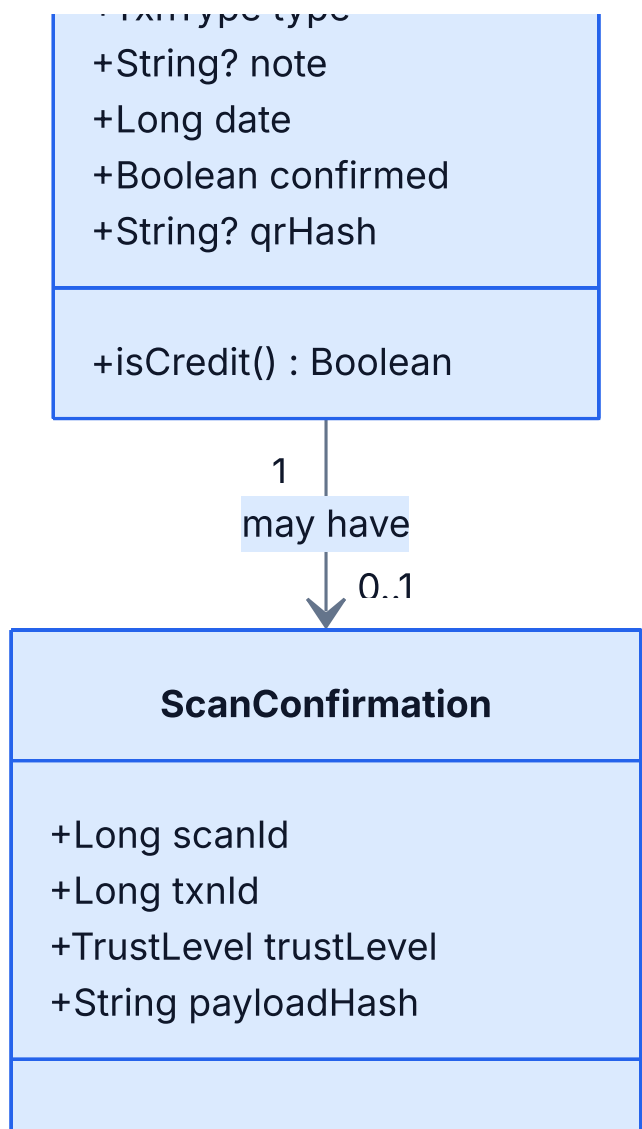


Figure 3.1 — UML Class Diagram with entity relationships

Key Relationships

From	To	Type
Customer	Transaction	One-to-Many
Transaction	ScanConfirmation	One-to-Optional
ViewModel	Repository	Dependency

4. Use Case Diagram

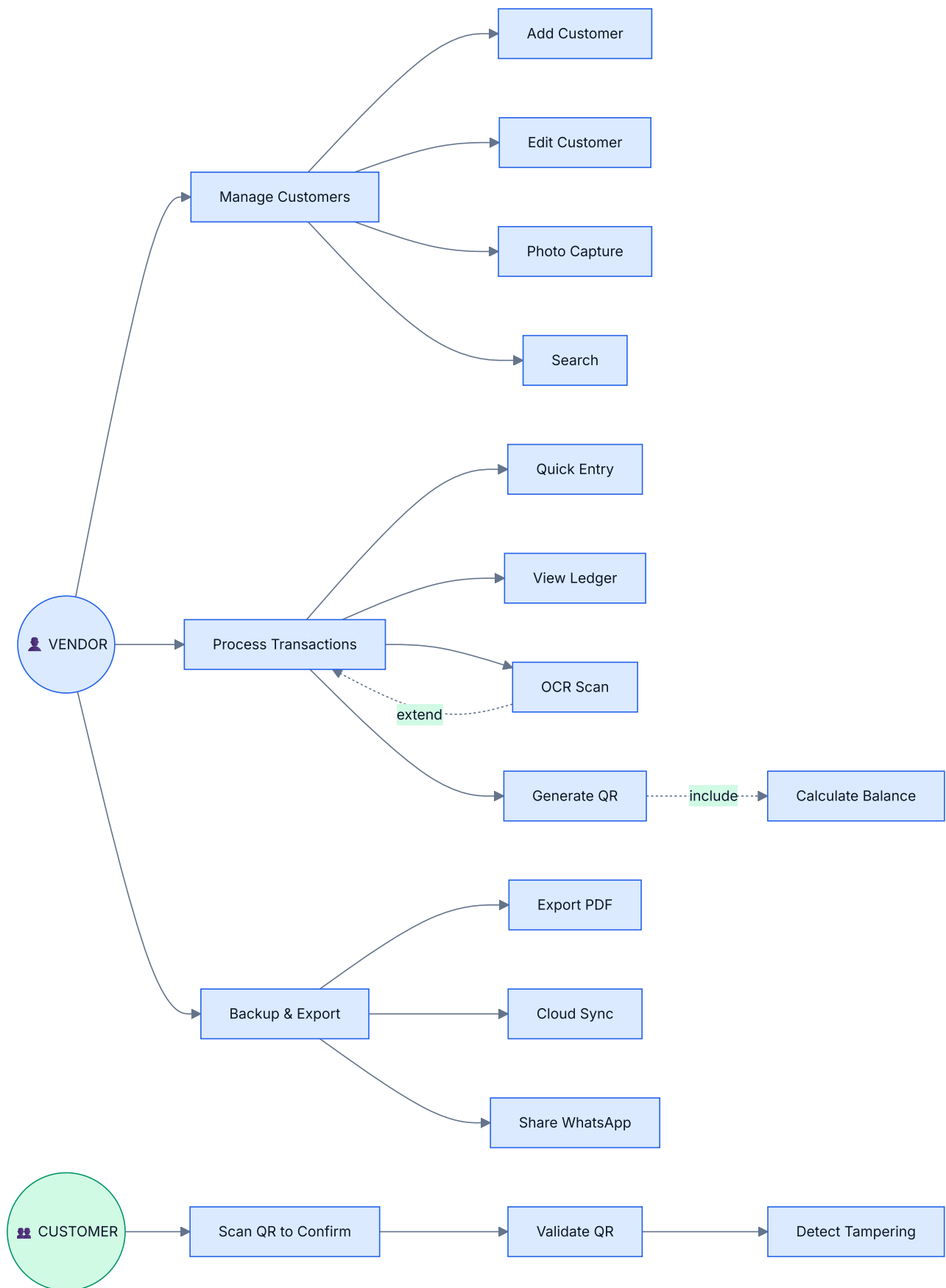


Figure 4.1 — Use Case Diagram: Vendor (primary) and Customer (secondary) actors

5. Sequence Diagram — Add Transaction

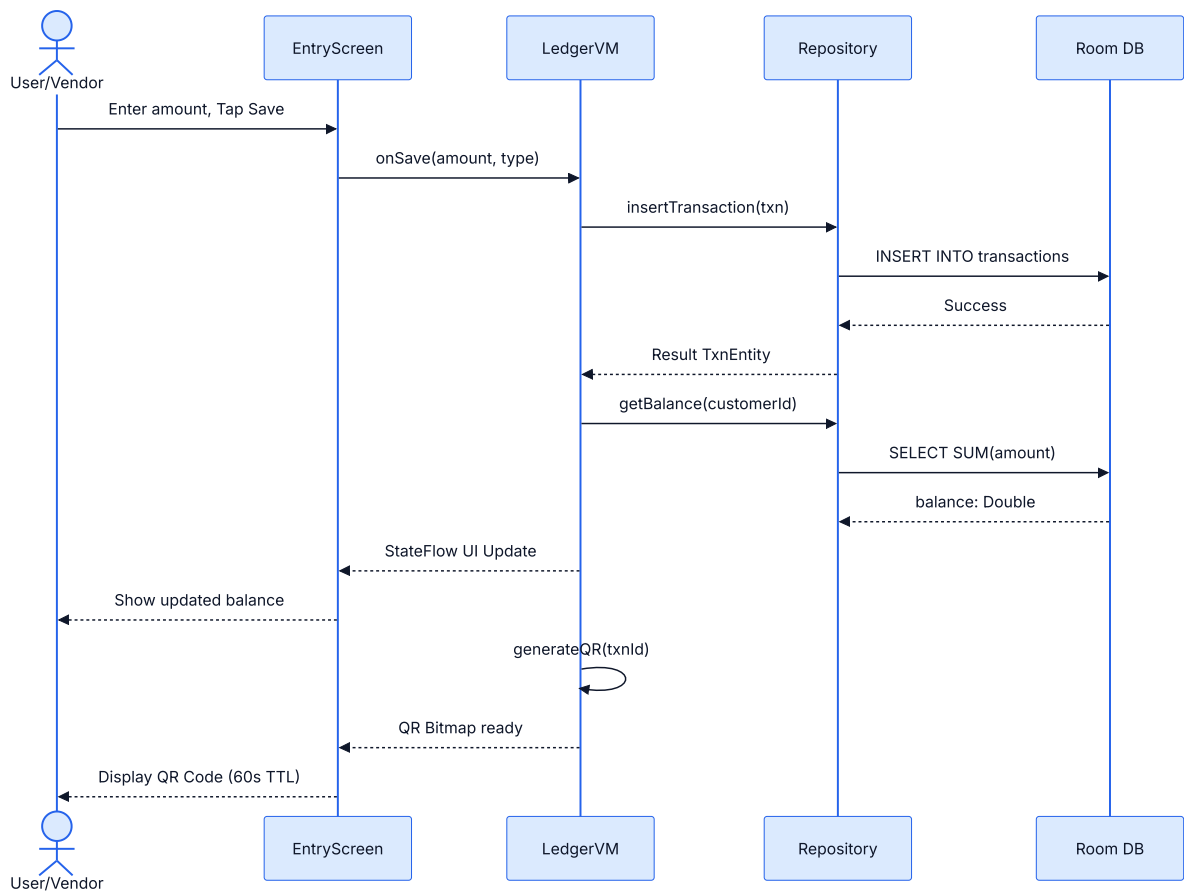


Figure 5.1 — Sequence diagram for the Add Transaction flow

6. Data Flow Diagram (Level 1)

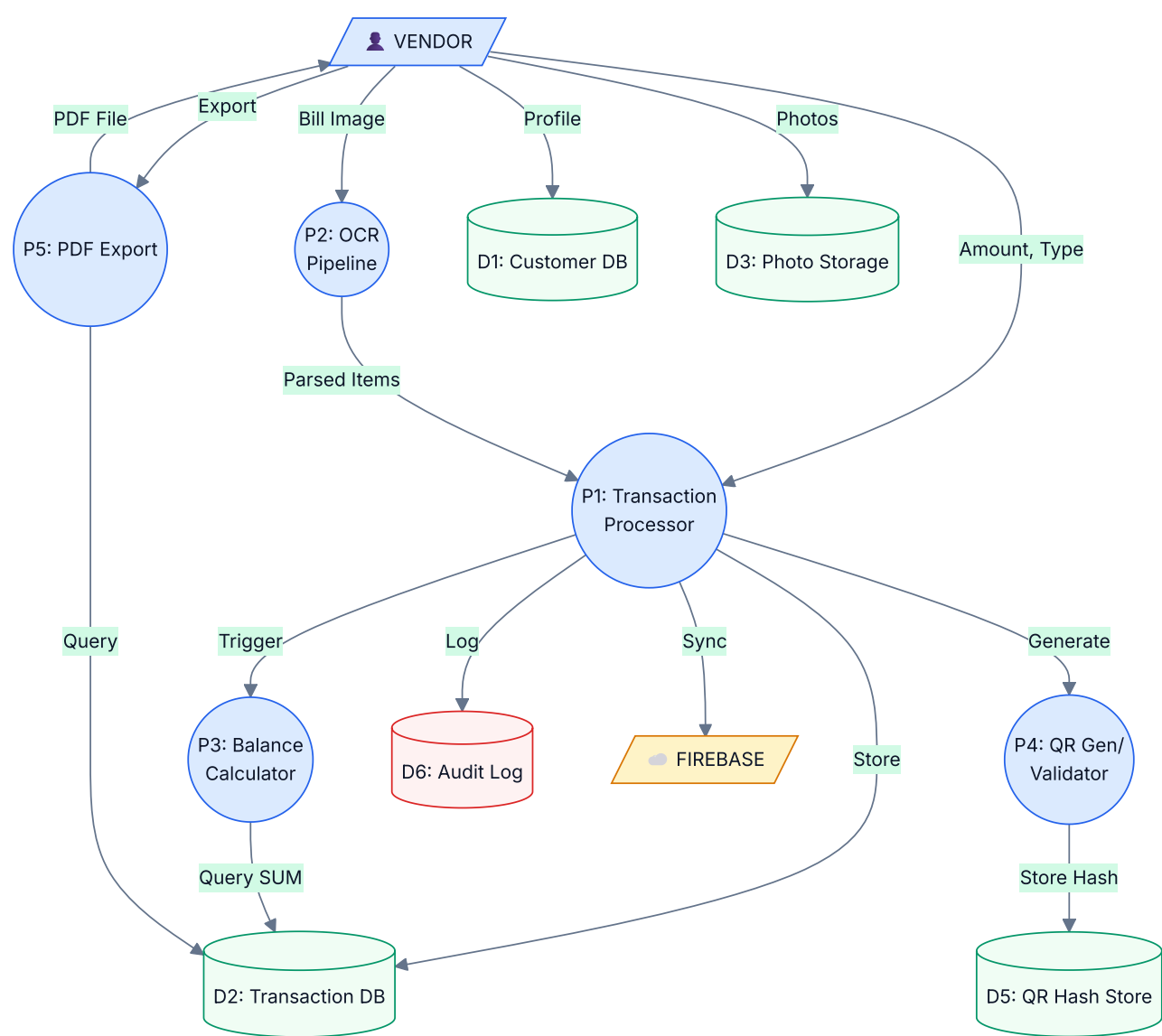
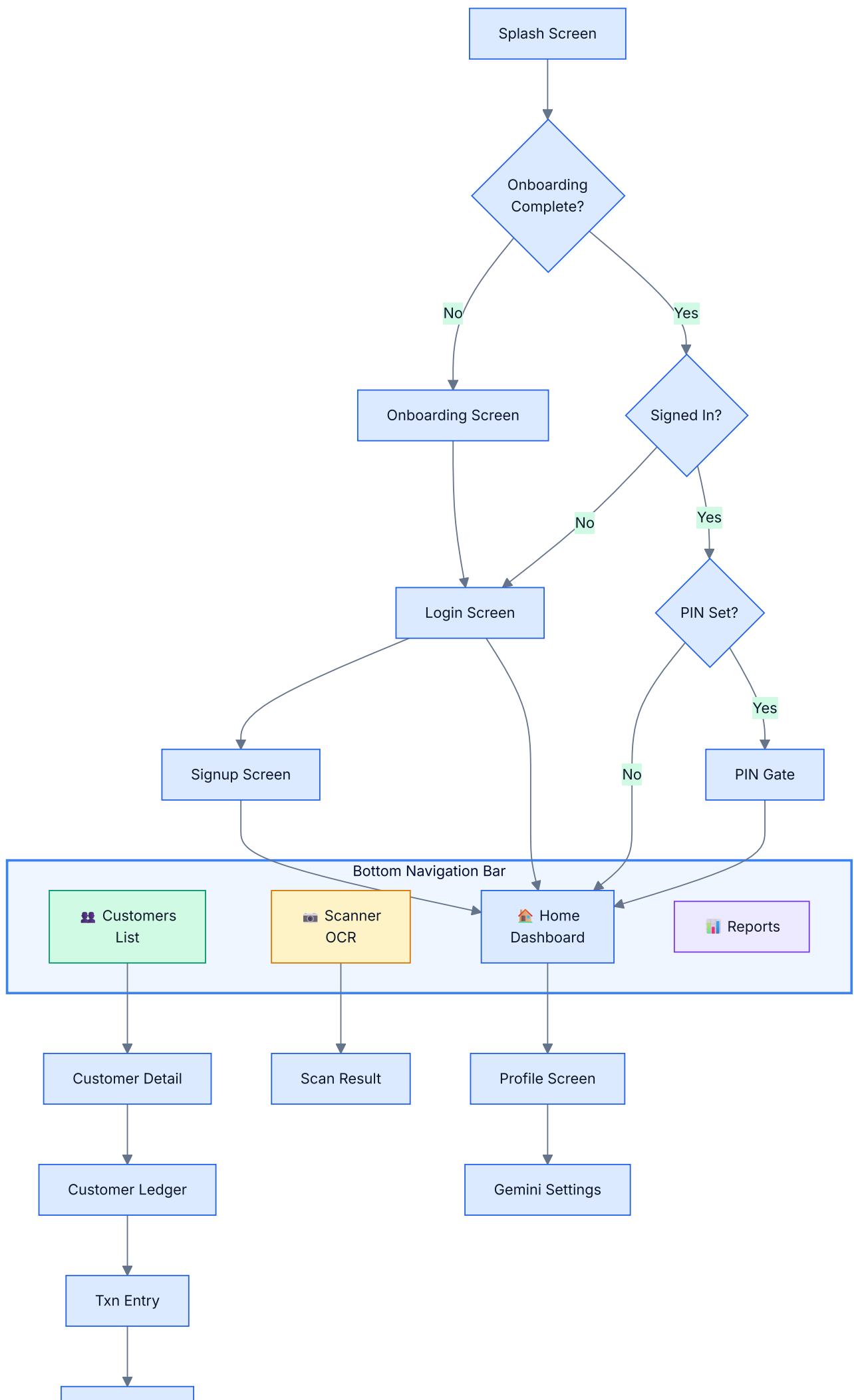


Figure 6.1 — Level 1 Data Flow Diagram with data stores

7. App Navigation Flow



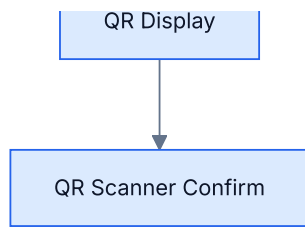
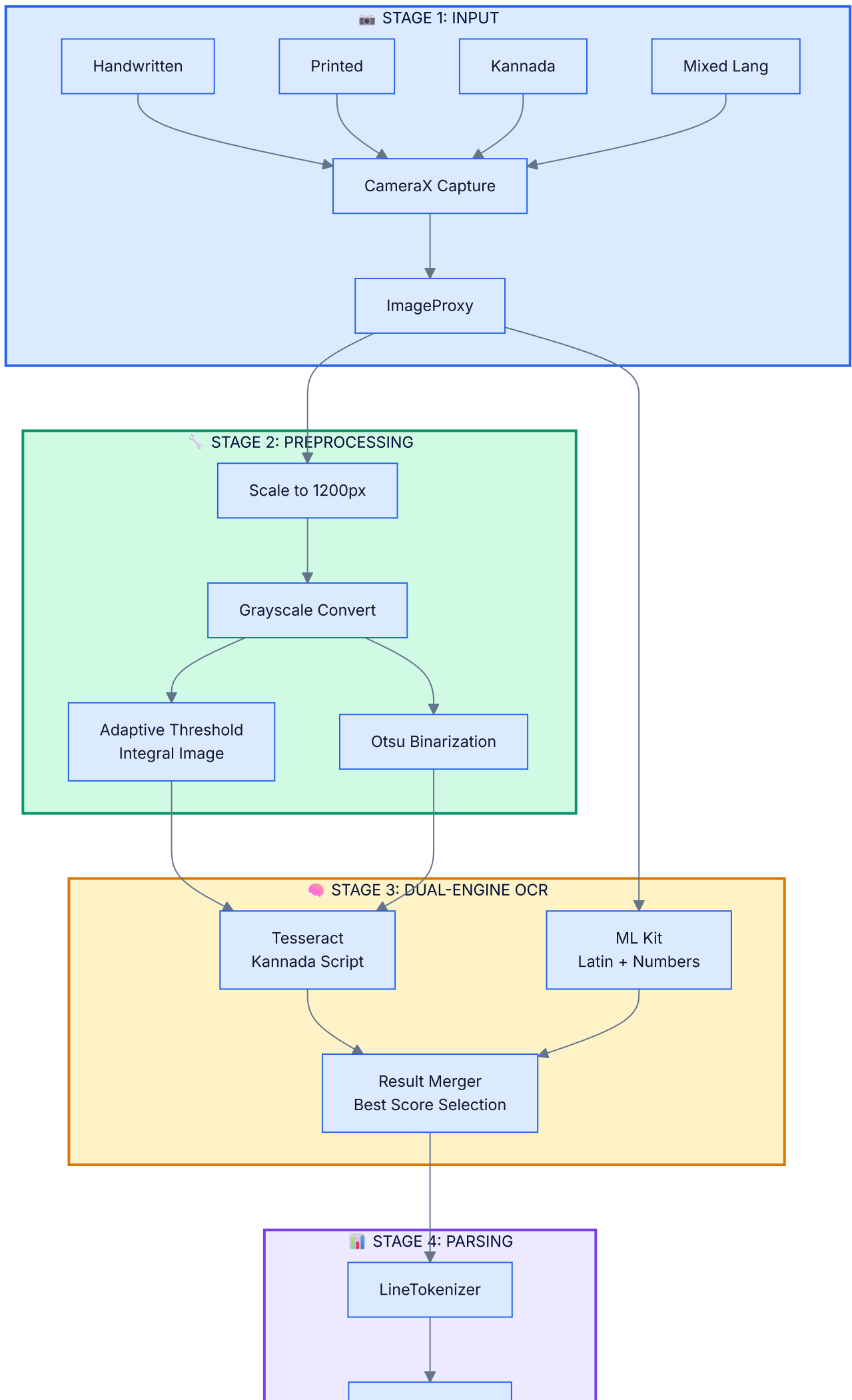


Figure 7.1 — Complete app navigation flow with auth gates

8. OCR Pipeline Architecture



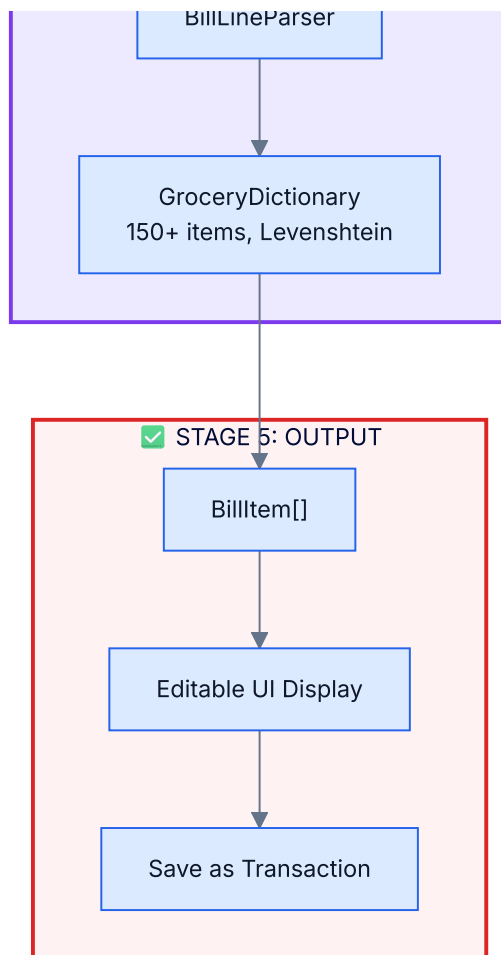


Figure 8.1 — Multi-engine OCR pipeline with preprocessing and parsing

9. QR Confirmation System

● QR GENERATION — Vendor

Transaction Saved



Build Payload
txnId+amount+type+nonce



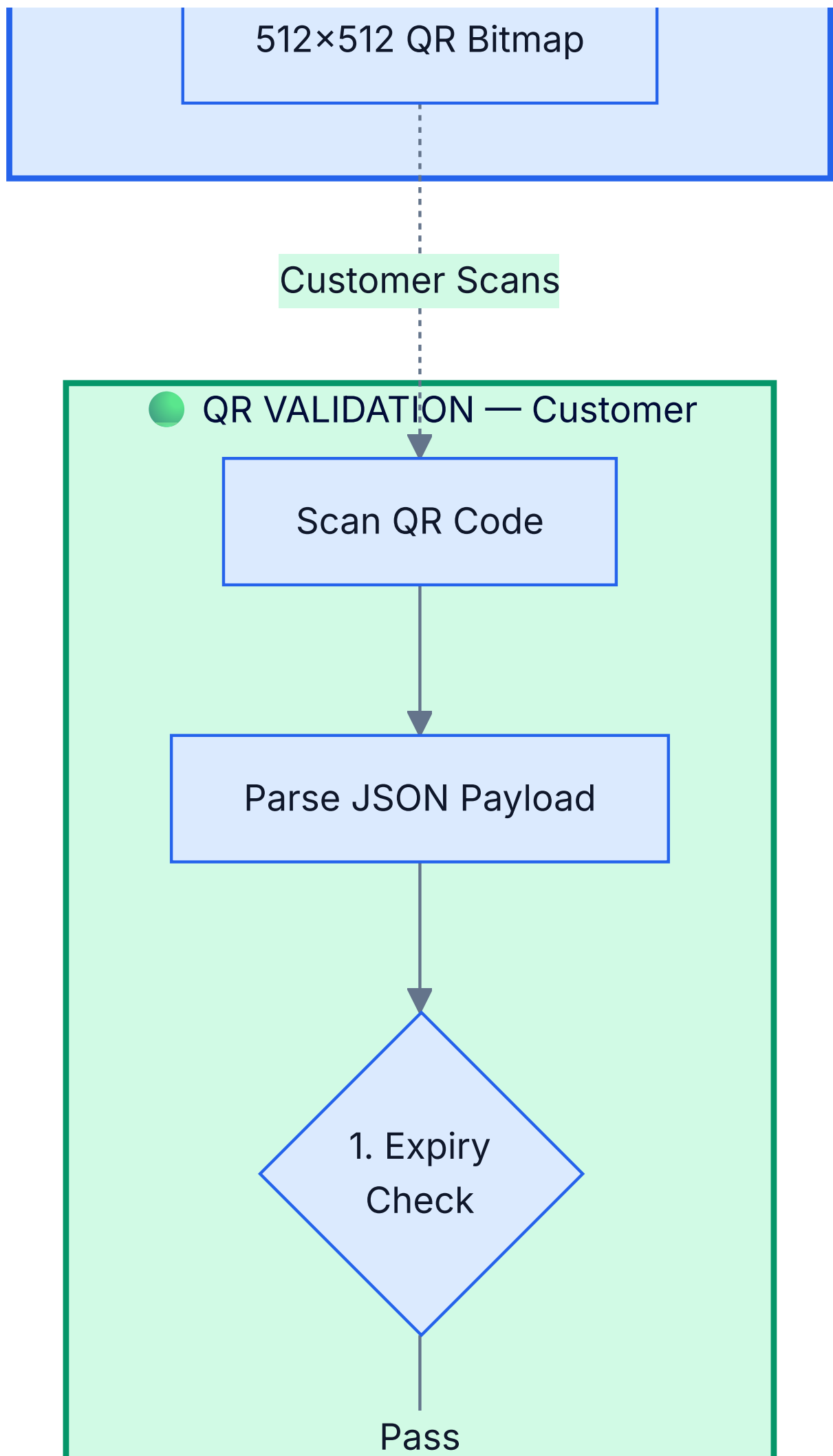
SHA-256 Hash
with SECRET_KEY

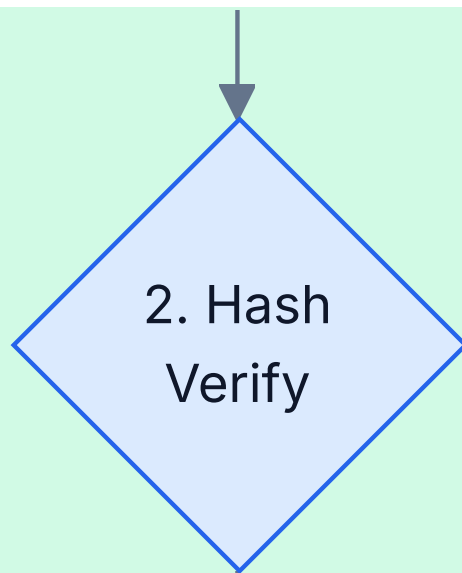


Set Expiry
timestamp + 60s TTL

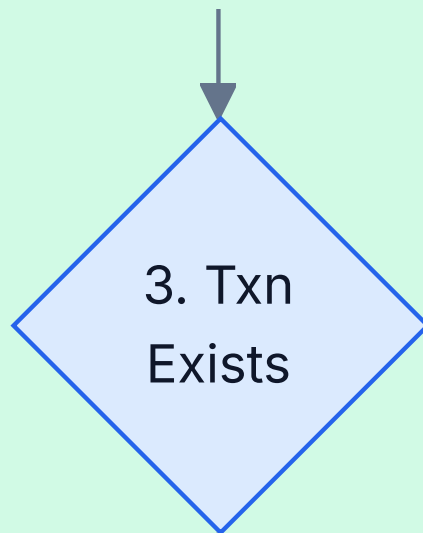


ZXing Encode

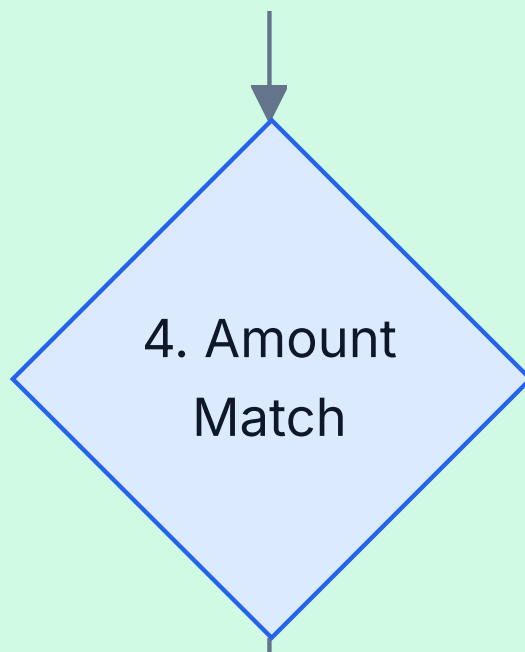


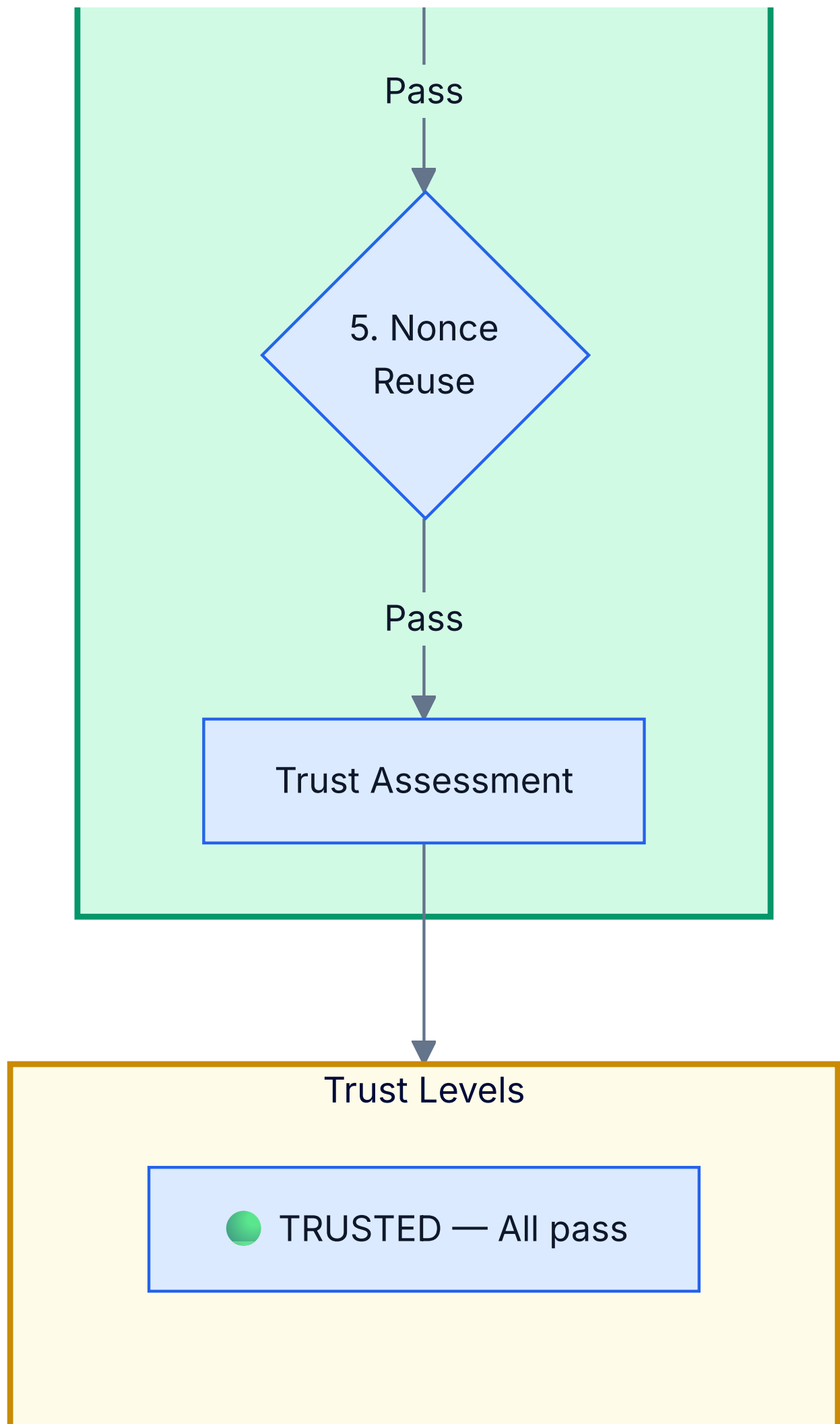


Pass



Pass





● VERIFIED — 3/4 pass

● CAUTION — 2/4 pass

● UNTRUSTED — <2 pass

Figure 9.1 — QR generation and multi-step validation flow

10. Security & Tamper Protection

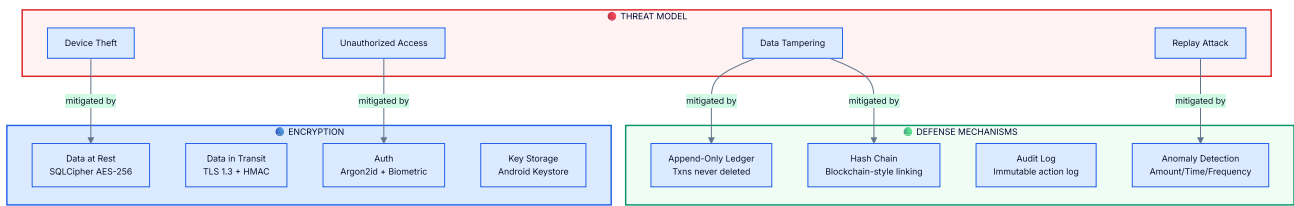


Figure 10.1 — Security threat model with defense and encryption layers

11. Entity-Relationship Diagram

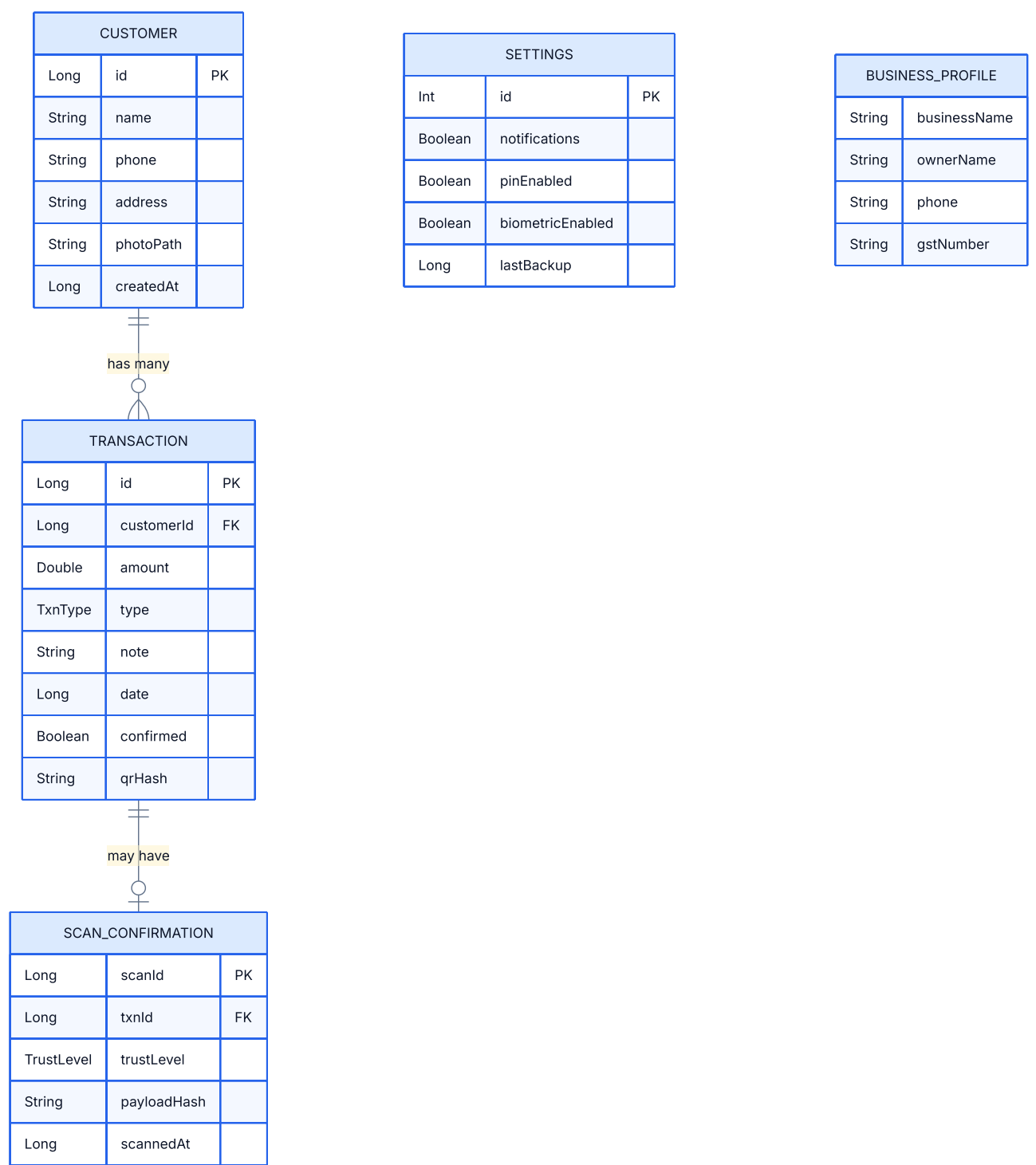


Figure 11.1 — Database entity-relationship diagram with crow's foot notation

Table Summary

Table	Key Fields	Relationships
Customer	id (PK), name, phone, photoPath	1 → * Transaction
Transaction	id (PK), customerId (FK), amount, type	1 → 0..1 ScanConfirmation
Settings	id (PK), pinEnabled, lastBackup	Standalone

ScanConfirmation	scanId (PK), txnId (FK), trustLevel	Belongs to Transaction
------------------	-------------------------------------	------------------------

12. Project Structure

```
app/src/main/java/com/nammasanthe/ledger/
├─ MainActivity.kt           # Entry point, Navigation host
├─ data/
│  ├─ database/AppDatabase.kt # Room database
│  ├─ entity/                # Customer, TxnEntity, Confirmation
│  └─ dao/                   # CustomerDao, TransactionDao
├─ repository/
│  ├─ LedgerRepository.kt    # Main CRUD operations
│  └─ DataExportManager.kt   # PDF export + sharing
├─ viewmodel/
│  ├─ LedgerViewModel.kt     # Ledger business logic
│  ├─ OcrViewModel.kt       # OCR pipeline control
│  └─ ConfirmationViewModel.kt # QR generation/scanning
├─ ui/screens/              # All Compose screens
├─ sync/
│  ├─ FirebaseAuthManager.kt # Auth (signup/signin)
│  └─ FirestoreSyncManager.kt # Firestore cloud sync
├─ security/                # QR Gen, Validate, Scan
├─ ocr/                     # Pipeline, Tesseract, ML Kit
│  ├─ OcrPipeline.kt        # Dual-engine orchestrator
│  ├─ TesseractOcr.kt       # Kannada OCR
│  ├─ BillParser.kt         # Bill item extraction
│  └─ GroceryDictionary.kt  # 150+ multilingual items
├─ gemini/GeminiService.kt  # Gemini API OCR
└─ util/                    # Photo, PDF utilities
```

13. Technology Stack

Layer	Technology	Purpose
UI	Jetpack Compose	Declarative Android UI
State	Kotlin StateFlow	Reactive state management
Architecture	MVVM + Repository	Clean separation of concerns
Database	Room (SQLite)	Offline-first persistence
OCR	Tesseract + ML Kit	Dual-engine text recognition
AI/ML	Gemini 2.5 Flash	Advanced OCR + translation
QR	ZXing	QR generation/scanning
PDF	iText7	Report generation
Auth	Firebase Auth	Email/password auth
Cloud	Firestore	Cloud database sync
Camera	CameraX	Photo + QR scanning

